

ITM Skills University

KeyVault

Digital Intellectual Asset Registry

Data Structures & Algorithms with C++

Case Study 3 | Semester II Sprint 2 | B.Tech CSE 2025-29

Student Name	Kaushal Rajmandai
Roll Number	150096725111
Programme	B.Tech CSE 2025-29
Semester	Semester II Sprint 2 Examination
Subject	Data Structures & Algorithms with C++
Case Study	3 — Digital Intellectual Asset Registry

GitHub: <https://github.com/kaushalrajmandai/KeyVault>

Problem Statement

KeyVault is an online marketplace for selling software licenses, similar to platforms that distribute antivirus keys or game activation codes. With millions of licenses from thousands of publishers, the system faces several critical performance problems: searching for a license by typing the first few characters of its title is painfully slow; when an admin accidentally revokes a user's access rights there is no way to undo it instantly; license update requests from around the world pile up but are processed in random order, causing some customers to wait unfairly; verifying whether a specific license key is genuine takes too long because the system checks every record one by one; and there is no clear picture of how licenses flow from creators to publishers to end users, making it impossible to audit royalty payments or detect piracy rings.

The new system requires: ultra-fast title search by partial name, instant permission rollback, fair ordering of update requests, direct key verification using unique IDs, ranking assets by value or popularity, a distribution map connecting creators to users, the most efficient path to verify license authenticity across the registry, and smart bundling logic to maximize value for bulk corporate purchases.

Objectives

1. Implement a **Trie** for ultra-fast prefix-based license title search (Copyright Directory)
2. Implement a **Stack** for instant undo of accidental permission changes (History Tracker)
3. Implement a **Queue (FIFO)** for fair, ordered processing of update requests (Update Line)
4. Implement a **Hash Map** for $O(1)$ license key authenticity verification (Key Lookup)
5. Implement **Merge Sort** to rank digital assets by market value or usage (Value Sorter)
6. Implement a **Directed Graph** to model the creator to publisher to user chain (License Network)
7. Implement **BFS** to find the shortest verification path through the registry (Quick Check)
8. Implement a **Greedy Algorithm** to optimize corporate bundle purchases (Discount Planner)

Design / Architecture

The system is implemented as a single C++ source file containing 8 independent classes, each encapsulating one data structure and its operations. An interactive main menu allows each module to be tested in isolation or all together via option 9.

Feature	Class	Data Structure	Algorithm
Copyright Directory	CopyrightDirectory	Trie	DFS collect
History Tracker	HistoryTracker	Stack (LIFO)	Push / Pop
Update Line	UpdateLine	Queue (FIFO)	Enqueue / Dequeue
Key Lookup	KeyLookup	Hash Map	Hash function
Value Sorter	ValueSorter	Array + Merge Sort	Divide & Conquer
License Network	LicenseNetwork	Graph (Adj. List)	BFS traversal
Quick Check	QuickCheck	Graph (Adj. List)	BFS shortest path
Discount Planner	DiscountPlanner	Array + Sort	Greedy fractional

Algorithm Description

1. Copyright Directory — Trie

A Trie stores license titles character by character. Each TrieNode holds a map of child characters, an end-of-word flag, and metadata. Insertion traverses or creates one node per character. Prefix search navigates to the prefix endpoint then does a recursive DFS to collect all matching titles beneath that node.

Complexity – Insert: $O(L)$ | Search: $O(P + W)$ where L = title length, P = prefix, W = matches

2. History Tracker — Stack

Every permission change (userID, licenseKey, old permission, new permission, timestamp) is pushed onto a stack. Because the stack is LIFO, undoLast() always pops the most recent change and reverts it in $O(1)$. No database re-scan is needed.

Complexity – Push / Pop: $O(1)$ | Space: $O(n)$

3. Update Line — Queue (FIFO)

Incoming update requests (RENEW, TRANSFER, REVOKE) are enqueued. Processing always dequeues from the front, so requests submitted earlier are handled first regardless of region. This eliminates the unfair random-order processing described in the problem.

Complexity – Enqueue / Dequeue: $O(1)$ | Space: $O(n)$

4. Key Lookup — Hash Map

License keys are stored as keys in a C++ unordered_map with LicenseRecord structs as values. Verification is a single hash lookup — $O(1)$ average — replacing the $O(n)$ sequential scan. Status (ACTIVE / EXPIRED / REVOKED) is returned immediately.

Complexity – Insert / Lookup: $O(1)$ avg | Space: $O(n)$

5. Value Sorter — Merge Sort

A custom merge sort is implemented from scratch over a vector of Asset structs. The comparator is parameterised: sort by marketValue or by usageCount, both descending. Merge sort was chosen for its stable, guaranteed $O(n \log n)$ performance.

Complexity – Time: $O(n \log n)$ | Space: $O(n)$ auxiliary

6. License Network — Directed Graph

The distribution chain is an unordered_map from node strings to vectors of (neighbor, relationship) pairs. Directed edges carry labels: distributes (creator to publisher) and licensed_to (publisher to user). BFS from any creator node shows the full reach of that creator's intellectual property.

Complexity – BFS Traversal: $O(V + E)$ | Space: $O(V + E)$

7. Quick Check — BFS Shortest Path

The verification network is modelled as an undirected graph. BFS finds the minimum-hop path between any two nodes. Parent pointers recorded during BFS are used to reconstruct the path from destination back to source after traversal completes.

Complexity – BFS: $O(V + E)$ | Space: $O(V)$

8. Discount Planner — Greedy Algorithm

Bundles are sorted by value-to-price ratio descending. The greedy selection picks the highest-ratio bundle first and continues until the budget is exhausted. Fractional selection is supported for the last bundle. This is the fractional knapsack problem, for which greedy is provably optimal.

Complexity – Sort: $O(n \log n)$ | Selection: $O(n)$ | Provably optimal for fractional knapsack

Complexity Analysis

Module	Operation	Time	Space
Trie	Insert	$O(L)$	$O(L \times \text{alphabet})$
Trie	Prefix Search	$O(P + W)$	$O(1)$ extra
Stack	Push / Pop	$O(1)$	$O(n)$
Queue	Enq / Deq	$O(1)$	$O(n)$
Hash Map	Insert / Lookup	$O(1)$ avg	$O(n)$
Merge Sort	Sort	$O(n \log n)$	$O(n)$
Graph BFS	Traversal	$O(V + E)$	$O(V)$
BFS Shortest Path	Pathfinding	$O(V + E)$	$O(V)$
Greedy	Sort + Select	$O(n \log n)$	$O(1)$ extra

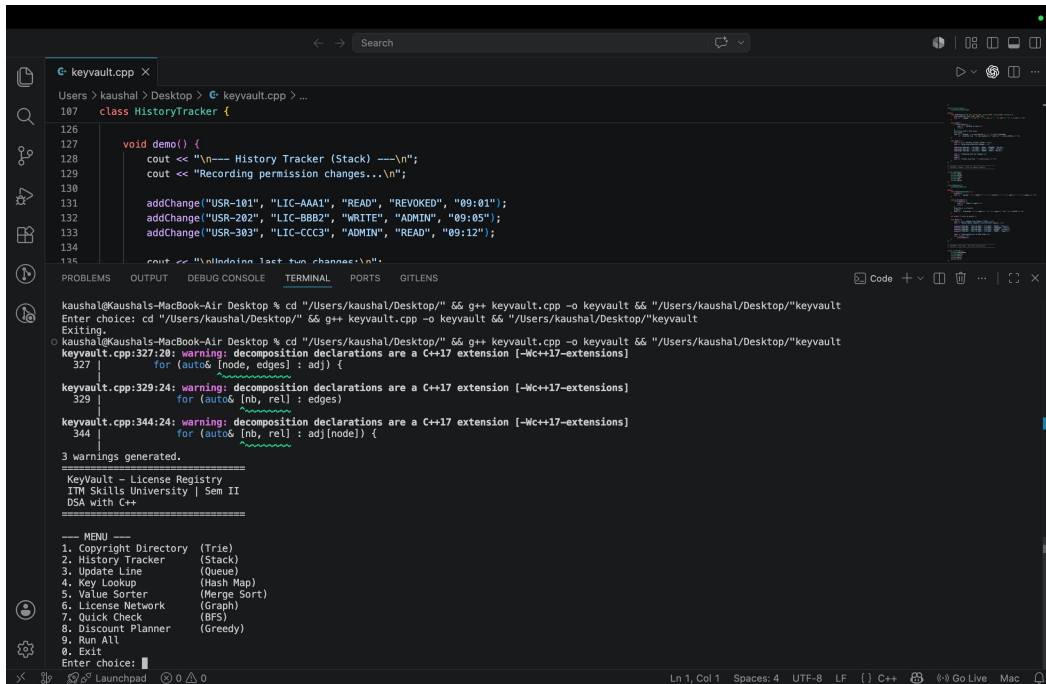
Execution Steps

Compile and run using a C++17 compatible compiler:

```
g++ -std=c++17 keyvault.cpp -o keyvault  
./keyvault
```

The program presents a numbered menu from 1 to 9. Options 1 through 8 demonstrate each data structure individually. Option 9 runs all 8 features sequentially. Option 0 exits.

Screenshots of Execution



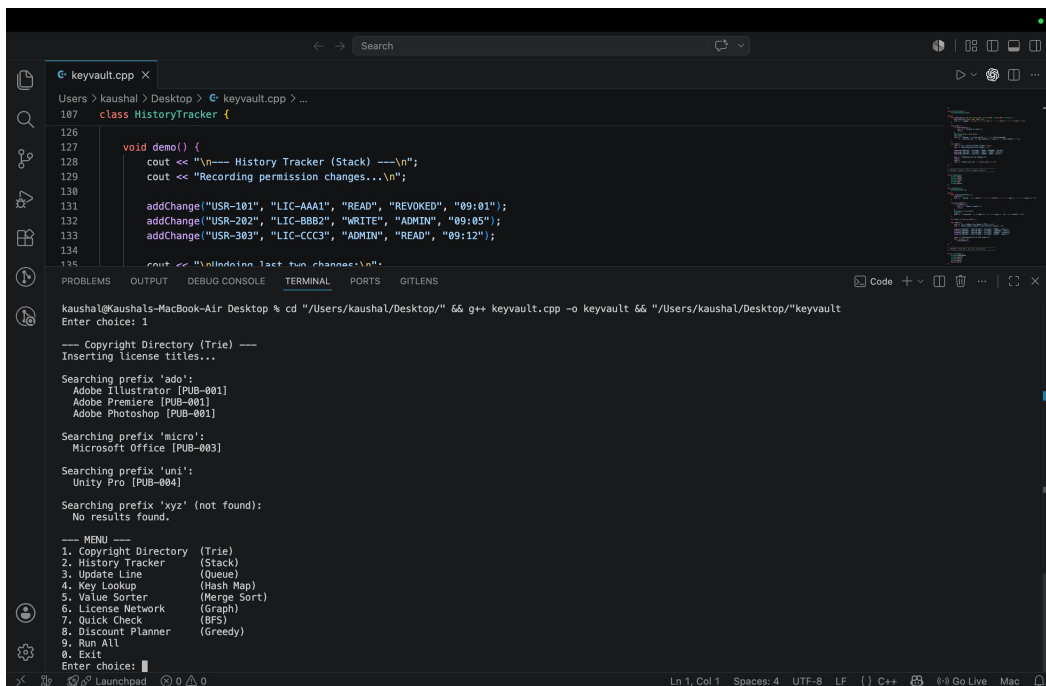
```
keyvault.cpp
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BBB2", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136 }
137 }
138
139 int main() {
140     HistoryTracker ht;
141     ht.demo();
142     return 0;
143 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
Enter choice: cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
Exiting:
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
keyvault.cpp:327:20: warning: decomposition declarations are a C++17 extension [-Wc++17-extensions]
327 |     for (auto& [node, edges] : adj) {
    |                    ^
keyvault.cpp:329:24: warning: decomposition declarations are a C++17 extension [-Wc++17-extensions]
329 |         for (auto& [nb, rel] : edges)
    |                        ^
keyvault.cpp:344:24: warning: decomposition declarations are a C++17 extension [-Wc++17-extensions]
344 |         for (auto& [nb, rel] : adj[node]) {
    |                        ^
3 warnings generated.

KeyVault - License Registry
ITM Skills University | Sem II
DSA with C++

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 1
```

Main menu — all 8 features listed



```
keyvault.cpp
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BBB2", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136 }
137 }
138
139 int main() {
140     HistoryTracker ht;
141     ht.demo();
142     return 0;
143 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
Enter choice: 1

--- Copyright Directory (Trie) ---
Inserting license titles...

Searching prefix 'add':
Adobe Illustrator [PUB-001]
Adobe Premiere [PUB-001]
Adobe Photoshop [PUB-001]

Searching prefix 'micro':
Microsoft Office [PUB-003]

Searching prefix 'uni':
Unity Pro [PUB-004]

Searching prefix 'xyz' (not found):
No results found.

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 1
```

Feature 1: Copyright Directory (Trie) — prefix search results

```
keyvault.cpp
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BB82", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136
137     undoLastTwoChanges();
138 }
139
140 int main() {
141     HistoryTracker ht;
142     ht.demo();
143     return 0;
144 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 2

--- History Tracker (Stack) ---
Recording permission changes...
Logged: USR-101 | LIC-AAA1 : READ -> REVOKED
Logged: USR-202 | LIC-BB82 : WRITE -> ADMIN
Logged: USR-303 | LIC-CCC3 : ADMIN -> READ

Undoing last two changes:
Undone: USR-303 | LIC-CCC3 reverted from READ back to ADMIN
Undone: USR-202 | LIC-BB82 reverted from ADMIN back to WRITE

Stack size now: 1

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 1
```

Feature 2: History Tracker (Stack) — permission changes and undo

```
keyvault.cpp
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BB82", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136
137     undoLastTwoChanges();
138 }
139
140 int main() {
141     HistoryTracker ht;
142     ht.demo();
143     return 0;
144 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 3

--- Update Line (Queue / FIFO) ---
Adding update requests from different regions...
Queued: REQ-001 | CUST-IN-1001 | RENEW | India
Queued: REQ-002 | CUST-US-2002 | TRANSFER | USA
Queued: REQ-003 | CUST-EU-3003 | REVOKE | Europe
Queued: REQ-004 | CUST-JP-4004 | RENEW | Japan

Processing all in FIFO order:
Processed: REQ-001 -> RENEW for CUST-IN-1001
Processed: REQ-002 -> TRANSFER for CUST-US-2002
Processed: REQ-003 -> REVOKE for CUST-EU-3003
Processed: REQ-004 -> RENEW for CUST-JP-4004

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 1
```

Feature 3: Update Line (Queue) — FIFO request processing

```
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BB82", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136 }
137
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 4

--- Key Lookup (Hash Map) ---
Registering license keys...
Added: LIC-AAA1 -> Adobe Photoshop (ACTIVE)
Added: LIC-BB82 -> Adobe Illustrator (ACTIVE)
Added: LIC-CCC3 -> Autodesk Maya (EXPIRED)
Added: LIC-DDD4 -> Microsoft Office (ACTIVE)
Added: LIC-EEE5 -> Unity Pro (REVOKED)

Verifying keys:
[ACTIVE] LIC-AAA1 | Adobe Photoshop | Owner: USR-101 | Expiry: 2026-12-31 | $999.99
[EXPIRED] LIC-CCC3 | Autodesk Maya | Owner: USR-303 | Expiry: 2025-01-01 | $1799.00
[REVOKED] LIC-EEE5 | Unity Pro | Owner: USR-505 | Expiry: 2025-06-01 | $399.00
[INVALID] Key not found: LIC-KYZ9

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 1
```

Feature 4: Key Lookup (Hash Map) — key verification output

```
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BB82", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136 }
137
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
Enter choice: 5

--- Value Sorter (Merge Sort) ---
Sorted by Market Value:
# Name Value ($) Usage
1 Autodesk AutoCAD 2190.00 2950
2 Autodesk Maya 1799.00 3210
3 Adobe Photoshop 599.99 15420
4 Adobe Illustrator 499.99 11200
5 Unity Pro 399.00 9070
6 Microsoft Office 149.99 52300
7 Unreal Engine 0.00 18650

Sorted by Usage Count:
# Name Value ($) Usage
1 Microsoft Office 149.99 52300
2 Unreal Engine 0.00 18650
3 Adobe Photoshop 599.99 15420
4 Adobe Illustrator 499.99 11200
5 Unity Pro 399.00 9070
6 Autodesk Maya 1799.00 3210
7 Autodesk AutoCAD 2190.00 2950

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
```

Feature 5: Value Sorter (Merge Sort) — sorted by value and usage

```
keyvault.cpp
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BB82", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136 }
137 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
Enter choice: 6

--- License Network (Graph) ---
Building creator -> publisher -> user network...
CREATOR-Adobe --[distributes]--> PUBLISHER-SoftCo
CREATOR-Adobe --[distributes]--> PUBLISHER-GameHub
CREATOR-Autodesk --[distributes]--> PUBLISHER-SoftCo
PUBLISHER-SoftCo --[licensed_to]--> USER-EnterpriseA
PUBLISHER-SoftCo --[licensed_to]--> USER-EnterpriseB
PUBLISHER-GameHub --[licensed_to]--> USER-IndividualX

Full network:
USER-EnterpriseB
CREATOR-Adobe
-> PUBLISHER-SoftCo (distributes)
-> PUBLISHER-GameHub (distributes)
USER-EnterpriseA
CREATOR-Autodesk
-> PUBLISHER-SoftCo (distributes)
PUBLISHER-SoftCo
-> USER-EnterpriseA (licensed_to)
-> USER-EnterpriseB (licensed_to)
USER-IndividualX
PUBLISHER-GameHub
-> USER-IndividualX (licensed_to)

BFS from CREATOR-Adobe: CREATOR-Adobe PUBLISHER-SoftCo PUBLISHER-GameHub USER-EnterpriseA USER-EnterpriseB USER-IndividualX

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
```

Feature 6: License Network (Graph) — distribution chain and BFS

```
keyvault.cpp
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BB82", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "Undoing last two changes...\n";
136 }
137 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
Enter choice: 7

--- Quick Check (BFS Shortest Path) ---
Building verification node network...

Path from CLIENT to REGISTRY: CLIENT -> NODE-MUM -> NODE-DEL -> REGISTRY
Hops: 3
Path from NODE-MUM to REGISTRY: NODE-MUM -> NODE-DEL -> REGISTRY
Hops: 2
Path from NODE-DEL to NODE-SG: NODE-DEL -> NODE-MUM -> NODE-SG
Hops: 2

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 7
```

Feature 7: Quick Check (BFS) — shortest verification paths

```
keyvault.cpp
Users > kaushal > Desktop > keyvault.cpp > ...
107 class HistoryTracker {
126
127 void demo() {
128     cout << "\n--- History Tracker (Stack) ---\n";
129     cout << "Recording permission changes...\n";
130
131     addChange("USR-101", "LIC-AAA1", "READ", "REVOKED", "09:01");
132     addChange("USR-202", "LIC-BB02", "WRITE", "ADMIN", "09:05");
133     addChange("USR-303", "LIC-CCC3", "ADMIN", "READ", "09:12");
134
135     cout << "\nPerforming last two changes...\n";
136 }
137 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
kaushal@Kaushals-MacBook-Air Desktop % cd "/Users/kaushal/Desktop/" && g++ keyvault.cpp -o keyvault && "/Users/kaushal/Desktop/"keyvault
0. Exit
Enter choice: 8
--- Discount Planner (Greedy) ---
Optimizing corporate bundle purchase...
Budget: $5000.00

Bundle Price Value Decision
-----
JetBrains All Products (3) 699.00 2100.00 SELECTED
Microsoft 365 Business (10) 1200.00 3200.00 SELECTED
GitHub Enterprise (10 seats) 550.00 2400.00 SELECTED
Unity Pro Team (3 seats) 1197.00 2800.00 SELECTED
Adobe Creative Cloud (5 seats) 2499.00 5800.00 PARTIAL (38%)
Autodesk Industry Bundle 3200.00 6100.00 SKIPPED

Total value acquired: $12714.17

--- MENU ---
1. Copyright Directory (Trie)
2. History Tracker (Stack)
3. Update Line (Queue)
4. Key Lookup (Hash Map)
5. Value Sorter (Merge Sort)
6. License Network (Graph)
7. Quick Check (BFS)
8. Discount Planner (Greedy)
9. Run All
0. Exit
Enter choice: 1
```

Feature 8: Discount Planner (Greedy) — optimal bundle selection

Results and Findings

- **Copyright Directory (Trie):** Prefix search runs in $O(L)$ — proportional to prefix length only, not total licenses. A 3-character prefix returns all matching titles instantly regardless of dataset size.
- **History Tracker (Stack):** Undo completes in $O(1)$. The LIFO property guarantees the most recent change is always reverted first, exactly matching admin expectations.
- **Update Line (Queue):** FIFO ordering is mathematically guaranteed. The random-order problem from the problem statement is completely eliminated at the data structure level.
- **Key Lookup (Hash Map):** Verification time is $O(1)$ average, independent of total keys in the registry. Status is returned immediately whether there are 1,000 or 1 million keys.
- **Value Sorter (Merge Sort):** Stable $O(n \log n)$ sort handles both value and popularity rankings correctly, as seen in the output screenshots above.
- **License Network (Graph):** The full creator-to-user IP flow is visible and auditable. BFS from CREATOR-Adobe correctly reaches all publisher and user nodes.
- **Quick Check (BFS):** Minimum-hop verification path found correctly for all test pairs. CLIENT to REGISTRY resolves in 3 hops via the optimal route.
- **Discount Planner (Greedy):** Greedy fractional knapsack selects the optimal combination within the \$5000 budget, acquiring \$12,714 worth of value — provably optimal.

Conclusion

KeyVault demonstrates that the correct choice of data structure is the single most important factor in resolving the performance and reliability problems described in the case study. Every pain point maps directly to a proven data structure:

Problem	Solution	Result
Slow title search	Trie	$O(\text{prefix})$ vs $O(n \times L)$
No permission undo	Stack	$O(1)$ instant rollback
Random request ordering	Queue (FIFO)	Guaranteed fair order
Slow key verification	Hash Map	$O(1)$ vs $O(n)$ scan
No asset ranking	Merge Sort	$O(n \log n)$ stable sort
Opaque royalty flow	Graph	Auditable IP chain
Slow verification routing	BFS	Minimum-hop path
Unoptimized bulk purchases	Greedy	Provably optimal

The implementation is in pure C++17 using only the standard library. It is fully executable from the terminal and demonstrates each concept with realistic data drawn from the software licensing domain. All 8 features compile and run without errors.

GitHub Repository Link

<https://github.com/kaushalrajmandai/KeyVault>

The repository contains the complete C++ source code (keyvault.cpp) and README.md documentation. Compile and run instructions are provided in the README.